

Fondements informatiques I

Cours 3: Structures itératives

Sorina Ionica `sorina.ionica@uvsq.fr`

Bilal FAYE `bilal.faye@uvsq.fr`

Les structures itératives (boucles)

Pourquoi utiliser une boucle ?

Une boucle permet de **répéter automatiquement** un ensemble d'instructions plusieurs fois.

Exemple

On veut afficher Programmer c'est cool ! **100 fois**.

Sans boucle, il faudrait écrire 100 fois :

```
print("Programmer c'est cool !")
```

→ Ce serait long, répétitif et source d'erreurs.

Idée

Écrire une seule fois l'instruction et demander au programme de la répéter.

L'itération

Une itération

Une itération consiste à **répéter un traitement** plusieurs fois.

Les éléments que l'on peut retrouver dans une boucle :

- une **variable de contrôle** : indique l'état ou la progression de la boucle ;
- une **condition de sortie** : détermine quand arrêter la boucle ;
- un **itérateur** : fait évoluer la variable de contrôle au fil des répétitions.

Deux structures itératives en Python

`for`

`while`

Ces éléments ne sont pas nécessairement présents de la même manière dans toutes les boucles.

L'instruction for

Avec une boucle for

La boucle for permet de répéter un bloc d'instructions **pour chaque élément d'une séquence**.

Syntaxe

```
for nom_variable in sequence:  
    instruction 1  
    instruction 2  
    ...  
    instruction n
```

- **Variable de contrôle** : `nom_variable` prend successivement les valeurs de la séquence.
- **Itérateur** : le passage à l'élément suivant fait évoluer la variable de contrôle.
- **Condition de sortie** : la boucle s'arrête lorsque tous les éléments de la séquence ont été parcourus.

L'instruction for avec range()

Pourquoi range() ?

range() retourne une séquence d'entiers, ce qui permet de contrôler le nombre de répétitions d'une boucle for.

À connaître

range(debut, fin) produit les valeurs de début à fin - 1.

La valeur de fin **n'est jamais incluse**.

Exemple

```
for i in range(1, 5):  
    print(i)
```

Affiche : 1 2 3 4

Les arguments de range() sont des entiers.

Les différentes formes de range()

Trois formes

```
range(fin)  
range(debut, fin)  
range(debut, fin, pas)
```

- `range(fin)` : commence à 0.
- `range(debut, fin)` : commence à debut.
- `range(debut, fin, pas)` : avance selon le pas.

Exemples

```
range(5) → 0, 1, 2, 3, 4  
range(2, 6) → 2, 3, 4, 5  
range(2, 6, 2) → 2, 4
```

Le pas dans range()

Le pas

Le troisième argument de `range()` indique le **pas d'avancement** entre deux valeurs.

S'il est omis, le pas vaut 1.

Exemples

```
range(1, 10, 2)    # 1, 3, 5, 7, 9  
range(10, 0, -2)   # 10, 8, 6, 4, 2
```

Le pas peut être positif ou négatif.

La variable de contrôle

Que devient `i` ?

À chaque itération, la variable de contrôle reçoit **la prochaine valeur produite par `range()`**.

Exemple

```
for i in range(1, 4):  
    print("i =", i)
```

Les valeurs de `i` sont successivement : 1, puis 2, puis 3.

Attention

Modifier `i` dans la boucle ne modifie pas les prochaines valeurs produites par `range()`.

Exercice : for et range()

Question

Que fait le programme suivant ?

```
n = int(input("Entrez un nombre : "))
resultat = 0
for i in range(1, n + 1):
    resultat += i
print("La somme vaut :", resultat)
```

Essayez de suivre les différentes valeurs de i .

Exemple : calculer une moyenne avec for

Objectif

Calculer la moyenne de plusieurs valeurs.

Exemple

```
notes = [12, 15, 14, 17]
somme = 0
for note in notes:
    somme += note
moyenne = somme / 4
print("Moyenne :", moyenne)
```

À retenir

À chaque itération, note prend successivement une valeur de notes.

La boucle while

Principe

La boucle `while` est utilisée lorsqu'on ne connaît pas à l'avance le nombre de fois que le traitement sera répété.

Syntaxe

```
while condition:
    instruction 1
    instruction 2
    ...
    instruction n
```

- Une **expression de contrôle** (*condition*), expression booléenne, contrôle l'exécution de la boucle.
- Cette expression est **testée avant chaque itération**.
- Une ou plusieurs instructions **indentées** sont répétées. Les instructions dans le bloc de la boucle sont exécutées si l'expression de contrôle reste satisfaite (**True**).

La boucle while : un exemple

Exemple

```
i = 1
while i < 10:
    print("i =", i)
    i = i + 1
print("Terminé")
```

- i est initialisée à 1.
- La condition $i < 10$ est **True** : la boucle commence et affiche la valeur de i.
- À chaque itération, i est augmentée de 1.
- Lorsque i vaut 10, la condition $i < 10$ devient **False** : la boucle se termine.

À retenir

La condition est vérifiée **avant chaque itération**. Lorsqu'elle devient **False**, le programme sort de la boucle et continue avec l'instruction suivante.

Exemple : boucle while

Quel est le résultat du code suivant ?

Code

```
somme = 0
i = 1
while i < 10:
    somme = somme + i
    i = i + 1
print(somme)
```

- Il faut inclure dans la boucle une instruction qui permet de **modifier l'état de l'expression de contrôle**.
- Il faut faire attention à **l'indentation** des instructions qui appartiennent à la boucle.

À retenir

La variable de contrôle doit évoluer pour permettre à la condition de devenir **False**.

Un programme plus complexe

Écrire un programme qui demande à l'utilisateur de deviner un nombre aléatoire entre 1 et 100.

Programme

```
import random # importe le module random
nombre_a_deviner = random.randint(1, 100)
reponse_utilisateur = 0
while reponse_utilisateur != nombre_a_deviner:
    reponse_utilisateur = int(input("Entrez un nombre:"))
    if reponse_utilisateur > nombre_a_deviner:
        print("Trop grand !")
    elif reponse_utilisateur < nombre_a_deviner:
        print("Trop petit !")
    else:
        print("Bravo !")
```

- `import random` permet d'utiliser les fonctions du module `random`.
- `random.randint(1, 100)` renvoie un entier aléatoire compris entre 1 et 100, bornes incluses.

Boucle for ou boucle while

À retenir

Toute boucle for peut être réécrite sous la forme d'une boucle while.

Avec for

```
for i in range(1, 6):  
    print("Hello world")
```

Le parcours de range() fait évoluer i automatiquement.

Avec while

```
i = 0  
while i < 5:  
    i += 1  
    print("Hello world")
```

Il faut gérer soi-même l'évolution de i.

Différence essentielle

for : le parcours est géré par la boucle
while : on gère la condition et l'évolution

Boucle for ou boucle while

À retenir

Toute boucle for peut être réécrite sous la forme d'une boucle while.

Avec for

```
for i in range(1, 6):  
    print("Hello world")
```

Le parcours de la séquence retournée par range() fait évoluer i automatiquement.

Avec while

```
i = 0  
while i < 5:  
    i += 1  
    print("Hello world")
```

Il faut faire évoluer i.

Différence essentielle

for : la variable de contrôle évolue automatiquement
while : le programmeur doit gérer la condition et l'évolution

Choisir entre for et while

Exercice

Donner la variante avec `while` du programme suivant.

Avec for

```
for a in "hello world":  
    print(a)
```

Boucles imbriquées

Une boucle peut être placée à l'intérieur d'une autre boucle : on parle alors de **boucles imbriquées**.

Ici, le joueur choisit à l'avance combien de parties il souhaite jouer.

Exemple

```
import random
nb_jeu = int(input("Combien de fois vous voulez jouer? "))
for i in range(0, nb_jeu):
    print("Le Jeu démarre!")
    nombre_a_deviner = random.randint(1, 100)
    reponse_utilisateur = 0
    while not reponse_utilisateur == nombre_a_deviner:
        reponse_utilisateur = int(input("Réponse:"))
        if reponse_utilisateur > nombre_a_deviner:
            print("Trop grand!")
        elif reponse_utilisateur < nombre_a_deviner:
            print("Trop petit !")
        else:
            print("Bravo!")
```

D'autres outils de contrôle : continue

Le mot-clé `continue` permet de passer prématurément à **l'itération suivante** de la boucle.

Les instructions qui suivent `continue` dans le bloc ne sont pas exécutées.

Exemple

```
for i in range(1, 21):
    if i % 2 == 0:
        continue
    print("résultat?")
    print(i, "est impair")

print("Terminé")
```

Que se passe-t-il ?

Si `i` est pair, `continue` est exécuté : **le reste de l'itération est ignoré** et la boucle passe à la suivante.

`print("résultat?")` ne sera donc jamais exécuté.

D'autres outils de contrôle : break

Le mot-clé `break` permet d'**arrêter immédiatement** l'exécution d'une boucle et de passer à l'instruction suivante.

Exemple

```
for i in range(1, 21):  
    if i % 2 == 0:  
        break  
    print(i, "est impair")  
print("Terminé")
```

Que se passe-t-il ?

La boucle commence avec `i = 1` : `1 est impair` est affiché.

À `i = 2`, la condition est vraie : `break` arrête **complètement la boucle**.

Le programme continue alors avec `print("Terminé")`.

Synthèse : boucles et instructions de contrôle

Exercice

```
mots = ["if", "while", "for", "break", "continue"]
for mot in mots:
    if mot == "while":
        continue
    elif mot == "break":
        break
    print(mot)
```

Questions

Quelles sont les valeurs affichées ?
À quel moment la boucle s'arrête-t-elle ?